A Note on Scaling Laws for Transformer Models — Part II

From a Small Training Recipe to a Large Model

μP and CompleteP

Junke Wang

Abstract

Can a training recipe tuned on a small model remain useful on a larger one? This note separates three changes: width, depth, and the training length. For width, μP coordinates initialization, forward multipliers, and learning rates so that hidden numbers and learned changes remain well scaled. We explain what "scale" means, derive the initialization rule, and distinguish input embeddings, hidden projections, and the final readout. CompleteP then adds a requirement for learning inside increasingly deep networks. For both, we derive AdamW's learning rate, ϵ, and weight decay. A noisy quadratic model then illustrates when a longer run or a smaller batch favors a smaller learning rate and why a final decay can help; measured horizon laws give the size of the correction, and fitted rules replace μP when it is not used. A joint width–depth rule table and a short pre-training check turn these arguments into a transfer procedure. Part III explains the optimizers themselves and extends the learning-rate rule to Muon.

Contents

1 What Should Transfer from the Small Model? 2

2 Mathematical Essentials 3

3 μP: Initialization, Forward Scaling, and Learning Rates 4

4 CompleteP: Depth Needs More than Stability 12

5 Learning Rate beyond Width and Depth 17

6 Summary: Transfer, Then Check 20

1 What Should Transfer from the Small Model?

Suppose a width-512, 8-layer Transformer trains well with AdamW. We want a width-2048, 32-layer version trained on more tokens. Can we copy the initialization and learning rate and start training?

A model stores trainable numbers called *weights*. A forward pass turns an input into a prediction through intermediate vectors, called *activations* or *features*. Training measures prediction error with a *loss* and adjusts the weights. An *optimizer* supplies the update rule; its learning rate multiplies the proposed parameter update. The resulting feature change also depends on the layer, its inputs, and the optimizer.

Each change alters a different part of the computation. A wider layer adds more terms in a matrix product. A deeper network adds more residual branches. A longer run takes more noisy updates. Keeping every numerical hyperparameter fixed therefore need not preserve the behavior that made the small model work.

Change	What must we control?	Main tool
Width	The scale of activations and learned feature changes	μP
Depth	Residual accumulation and nonlinear learning inside each block	CompleteP
Training length	Progress and noise accumulated over many updates	Horizon factor and schedule

These tools belong together because all three affect the quality and cost of training the models behind a scaling law. They do not make losses identical across model sizes. They help us avoid mistaking a poorly transferred recipe for a limitation of model scale.

We will keep notation separate: d is hidden width and L is the number of Transformer blocks. Reference width and depth are d_0, L_0, with multipliers

$$r := \frac{d}{d_0}, \qquad q := \frac{L}{L_0}. \tag{1}$$

A hyperparameter is a setting we choose, such as a learning rate. A *base* hyperparameter is tuned at this reference model. A transfer rule converts it into the initialization, forward multiplier, or optimizer setting used by the target model. Before examining width, we introduce the mathematical tools used in the argument.

Scope and assumptions. The worked recipe uses dense pre-normalized residual Transformers, a fixed vocabulary, an untied readout, and AdamW. Width means that hidden dimensions grow at fixed ratios; depth means more copies of the same block. The scalar and one-token examples explain the mechanisms, while cited theory supplies the broader scaling results. They are not proofs for arbitrary architectures. Tied embeddings, shared conditioning networks, AdaLN gates, and MoE routing need their own parameter inventory and checks. We distinguish three claims: well-scaled coordinates, nonvanishing nonlinear learning, and transfer of a useful hyperparameter range. The first alone does not establish the other two.

Reading route. For the main argument, read the mathematical essentials, then follow Sections 3 and 4: first control width, then depth. Section 6 collects the rules and the checks

needed before a larger run. The optimizer constants in Section 4.4 are implementation details to revisit when setting parameter groups. Section 5 treats the separate question of changing batch, training length, or schedule; it can be read after the width–depth argument.

2 Mathematical Essentials

We need two measurements and the relationship between them: variance describes random fluctuations, while RMS describes the coordinate scale of a vector.

2.1 Variance: how random terms add

For a random number Z, its expectation $\mathbb{E}[Z]$ is its probability-weighted average. Averaging respects addition and fixed multipliers: for random numbers Z, U and fixed numbers a, b, c,

$$\mathbb{E}[aZ + bU] = a\mathbb{E}[Z] + b\mathbb{E}[U], \qquad \mathbb{E}[c] = c. \tag{2}$$

The first rule needs no independence assumption. The second says that averaging a number that never changes gives that same number.

Write μ_Z for the mean of Z. Variance averages its squared distance from the mean; standard deviation σ_Z takes the square root:

$$\mu_Z = \mathbb{E}[Z], \qquad \mathrm{Var}(Z) = \mathbb{E}[(Z - \mu_Z)^2], \qquad \sigma_Z = \sqrt{\mathrm{Var}(Z)}. \tag{3}$$

Thus variance has squared units, while standard deviation has the same units as Z. Two consequences will let us analyze a layer.

Multiplying a random number. Multiplying Z by fixed c also multiplies its mean by c. Its deviation from that mean is therefore multiplied by c, and its squared deviation by c^2:

$$\begin{aligned} \mathrm{Var}(cZ) &= \mathbb{E}[(cZ - c\mu_Z)^2] \\ &= c^2\mathbb{E}[(Z - \mu_Z)^2] = c^2\,\mathrm{Var}(Z). \end{aligned} \tag{4}$$

Adding independent random numbers. Independence means that observing one number does not change the distribution of another. For independent Z_1, Z_2, write their deviations from their means as $\delta_j = Z_j - \mathbb{E}[Z_j]$, where $j = 1, 2$. Each deviation has mean zero. Independence lets us average their product separately:

$$\mathbb{E}[\delta_1\delta_2] = \mathbb{E}[\delta_1]\mathbb{E}[\delta_2] = 0. \tag{5}$$

Expanding the square in the variance definition now gives

$$\begin{aligned} \mathrm{Var}(Z_1 + Z_2) &= \mathbb{E}[(\delta_1 + \delta_2)^2] \\ &= \mathbb{E}[\delta_1^2] + 2\mathbb{E}[\delta_1\delta_2] + \mathbb{E}[\delta_2^2] \\ &= \mathrm{Var}(Z_1) + \mathrm{Var}(Z_2). \end{aligned} \tag{6}$$

Repeating this argument for m independent terms, and using the multiplier rule for fixed weights p_j, yields

$$\begin{aligned} \mathrm{Var}\left(\sum_{j=1}^{m} Z_j\right) &= \sum_{j=1}^{m} \mathrm{Var}(Z_j), \\ \mathrm{Var}\left(\sum_{j=1}^{m} p_j Z_j\right) &= \sum_{j=1}^{m} p_j^2\,\mathrm{Var}(Z_j). \end{aligned} \tag{7}$$

The symbol $\sum_{j=1}^{m}$ means "add the terms with indices 1 through m." The numbers p_j are fixed weights; they need not be equal. Without independence, the cross terms need not vanish.

2.2 RMS: the typical scale of a vector's coordinates

For a vector $\boldsymbol{x} = (x_1, \ldots, x_n)$ containing n numbers, root mean square is

$$\operatorname{rms}(\boldsymbol{x}) = \sqrt{\frac{x_1^2 + \cdots + x_n^2}{n}}. \tag{8}$$

Square the numbers, average them, and take the square root. Squaring prevents positive and negative entries from cancelling. The division by n lets us compare vectors with different numbers of entries. Throughout this note, *activation scale* means this RMS, computed across the hidden coordinates of one token. It is a measurement; applying an RMSNorm layer is a separate operation.

The L2 norm, written $\|\boldsymbol{x}\|_2$, omits the division by n:

$$\|\boldsymbol{x}\|_2 = \sqrt{x_1^2 + \cdots + x_n^2} = \sqrt{n}\ \operatorname{rms}(\boldsymbol{x}). \tag{9}$$

It can grow simply because the vector has more coordinates. Multiplying all coordinates by a fixed number c instead changes RMS by $|c|$, its absolute value. Directly from the definition,

$$\operatorname{rms}(c\boldsymbol{x}) = \sqrt{c^2 \frac{\sum_{j=1}^{n} x_j^2}{n}} = |c|\ \operatorname{rms}(\boldsymbol{x}). \tag{10}$$

2.3 Mean, variance, and RMS

For a random number Z, write its mean as $\mu = \mathbb{E}[Z]$. Variance measures the average squared distance from this mean. Expand its definition and use Eq. (2):

$$\begin{aligned} \operatorname{Var}(Z) &= \mathbb{E}[(Z-\mu)^2] \\ &= \mathbb{E}[Z^2 - 2\mu Z + \mu^2] \\ &= \mathbb{E}[Z^2] - 2\mu\mathbb{E}[Z] + \mu^2 \\ &= \mathbb{E}[Z^2] - \mu^2. \end{aligned} \tag{11}$$

The third line treats μ as a fixed number; the fourth substitutes $\mathbb{E}[Z] = \mu$.

RMS instead measures distance from zero. For a random number Z, its population RMS is, by definition, the square root of its mean square. This definition does not require a zero mean. Combining it with Eq. (11) gives

$$\boxed{\begin{aligned} &\operatorname{rms}(Z) := \sqrt{\mathbb{E}[Z^2]}, \\ &\quad \mu = 0 \quad \Longrightarrow \quad \operatorname{Var}(Z) = \mathbb{E}[Z^2], \\ &\quad \mu = 0 \quad \Longrightarrow \quad \operatorname{rms}(Z) = \sqrt{\operatorname{Var}(Z)}. \end{aligned}} \tag{12}$$

3 μP: Initialization, Forward Scaling, and Learning Rates

We will explain all three choices through the same linear operation:

$$\boldsymbol{y} = aW\boldsymbol{x}, \qquad y_i = a\sum_{j=1}^{d_{\text{in}}} W_{ij}x_j. \tag{13}$$

Here $\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}} \times 1}$ is one token's input feature, $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the weight matrix, $\boldsymbol{y} \in \mathbb{R}^{d_{\text{out}} \times 1}$ is the output feature, and $a \in \mathbb{R}_{>0}$ is the forward multiplier.

μP aims to preserve nontrivial feature learning as width changes: at corresponding tokens, layers, and training steps, hidden activations and their training-induced changes should neither vanish nor explode. It coordinates three choices to achieve this goal:

Choice	What it controls in this calculation	When it acts
Initialization	The starting entries of W	Before training
Forward scaling	The coefficient a multiplying $W\boldsymbol{x}$	Every forward pass
Learning rate	The size of each subsequent change to W	Every update

3.1 Initialization: control the first forward pass

Set up the layer and the goal. For a hidden projection the forward multiplier is $a = 1$. The ith output coordinate is the dot product of the input $\boldsymbol{x}$ with row i of W:

$$y_i = W_{i,:}\boldsymbol{x} = \sum_{j=1}^{d_{\text{in}}} W_{ij} x_j. \tag{14}$$

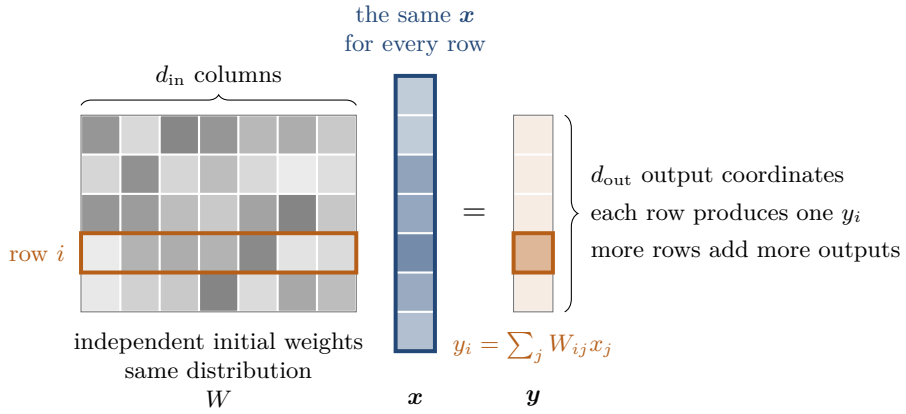


Figure 1: The product $\boldsymbol{y} = W\boldsymbol{x}$ for one token, with $d_{\text{in}} = 7$ and $d_{\text{out}} = 5$. Gray shades are random initial weights; blue shades are the fixed input. Each output y_i is row i of W times the shared $\boldsymbol{x}$, a sum of d_{in} terms. Longer rows (larger d_{in}) lengthen every sum; more rows (larger d_{out}) only add outputs of the same kind.

We now answer one question: how should the initial weight standard deviation σ_W depend on d_{in} so that $\text{rms}(\boldsymbol{y})/\text{rms}(\boldsymbol{x})$ does not change with width? Assume that the entries of W are independent and identically distributed, with mean 0 and variance σ_W^2. During this calculation, $\boldsymbol{x}$ is fixed and the expectation is only over the random initial weights.

Step 1: reduce the vector RMS to one output coordinate. Every output y_i uses the same input $\boldsymbol{x}$ but an independently drawn row of W. The coordinates y_i are therefore independent and have the same distribution. For a wide output vector, this gives:

$$\text{rms}(\boldsymbol{y}) = \sqrt{\frac{1}{d_{\text{out}}} \sum_{i=1}^{d_{\text{out}}} y_i^2} \approx \sqrt{\mathbb{E}_W[y_i^2]}. \tag{15}$$

For any output width, the exact identity is $\mathbb{E}_W[\mathrm{rms}(\boldsymbol{y})^2] = \mathbb{E}_W[y_i^2]$. The displayed approximation additionally uses concentration across output coordinates. Thus the variance rule does not require a large output width; only this replacement of a realized RMS by its typical value does.

Step 2: compute the expected square of one coordinate. Recall Eqs. (11) and (12). Applied to y_i, their relationship is

$$\mathbb{E}_W[y_i^2] = \mathrm{Var}_W(y_i) + \big(\mathbb{E}_W[y_i]\big)^2. \tag{16}$$

We now compute the two terms on the right separately. For the mean, each x_j is fixed and every W_{ij} has mean zero:

$$\mathbb{E}_W[y_i] = \mathbb{E}_W\left[\sum_{j=1}^{d_{\text{in}}} W_{ij}x_j\right] = \sum_{j=1}^{d_{\text{in}}} x_j \mathbb{E}_W[W_{ij}] = 0. \tag{17}$$

For the variance, multiplication by x_j contributes a factor x_j^2, and the independent terms add their variances:

$$\begin{aligned} \mathrm{Var}_W(y_i) &= \sum_{j=1}^{d_{\text{in}}} x_j^2 \,\mathrm{Var}_W(W_{ij}) \\ &= \sigma_W^2 \sum_{j=1}^{d_{\text{in}}} x_j^2 \\ &= d_{\text{in}}\sigma_W^2 \,\mathrm{rms}(\boldsymbol{x})^2. \end{aligned} \tag{18}$$

Substituting Eqs. (17) and (18) into Eq. (16) gives

$$\mathbb{E}_W[y_i^2] = d_{\text{in}}\sigma_W^2 \,\mathrm{rms}(\boldsymbol{x})^2. \tag{19}$$

Step 3: return to the vector RMS and solve for σ_W. Substituting Eq. (19) into Eq. (15) gives

$$\mathrm{rms}(\boldsymbol{y}) \approx \sqrt{d_{\text{in}}}\,\sigma_W \,\mathrm{rms}(\boldsymbol{x}), \qquad \frac{\mathrm{rms}(\boldsymbol{y})}{\mathrm{rms}(\boldsymbol{x})} \approx \sqrt{d_{\text{in}}}\,\sigma_W. \tag{20}$$

To keep this ratio equal to a width-independent constant $g > 0$, choose

$$\sqrt{d_{\text{in}}}\,\sigma_W = g \quad \Longrightarrow \quad \boxed{\sigma_W = \frac{g}{\sqrt{d_{\text{in}}}}.} \tag{21}$$

The input width appears because every y_i sums d_{in} terms. The output width does not appear because adding rows creates more coordinates; it does not add terms to any one coordinate.

This fan-in rule is also used by standard parametrization. It is only the initialization part of μP: training can make a weight matrix correlated with its input, which breaks the independence assumed above (Sections 3.2 and 3.3).

3.2 Learning rates: control this layer's own feature change

Section 3.1 fixed the output scale before training. Training then changes W, and the output change it causes must also have a width-independent RMS. Here we hold the input fixed and change only the matrix. This isolates the feature change created by the layer's own weight update, whose size we control through the learning rate.

Step 1: write the exact quantity that the learning rate controls. Let t count training steps. Each step moves every weight against an update direction U, scaled by the learning rate $\eta > 0$:

$$W_{ij}^{t} = W_{ij}^{t-1} - \eta\, U_{ij}. \tag{22}$$

We use AdamW and first isolate its gradient-driven update, leaving weight decay to Section 4.4. It computes U from the gradient $G := \partial\mathcal{L}/\partial W$, evaluated at W^{t-1}, by dividing a running average of gradients, entry by entry, by the square root of a running average of squared gradients. To isolate this layer's own effect, hold $\boldsymbol{x}$ fixed and change only W:

$$\Delta y_{i,\text{own}} := \sum_{j=1}^{d_{\text{in}}} (W_{ij}^{t} - W_{ij}^{t-1}) x_j = -\eta \sum_{j=1}^{d_{\text{in}}} U_{ij} x_j. \tag{23}$$

The subscript "own" excludes input changes produced by earlier layers. The learning rate multiplies this change directly, so we next determine how the sum changes with fan-in.

Step 2: estimate how the update grows with fan-in. Take the loss on this one token. For this calculation only, use the current gradient in place of the running averages and set $\epsilon = 0$, with zero gradient entries receiving zero update. Step 3 returns to batches and running averages. Since $y_i = \sum_j W_{ij} x_j$, the chain rule gives

$$G_{ij} = \frac{\partial\mathcal{L}}{\partial y_i}\frac{\partial y_i}{\partial W_{ij}} = \delta_i x_j, \qquad \delta_i := \frac{\partial\mathcal{L}}{\partial y_i}. \tag{24}$$

Row i of G is the input $\boldsymbol{x}$ multiplied by one number, so the gradient is aligned with its input. In this simplified update, each nonzero entry is divided by $|G_{ij}|$, so $U_{ij} = \operatorname{sign}(G_{ij}) = \operatorname{sign}(\delta_i)\operatorname{sign}(x_j)$. Substituting this into Eq. (23), with $\operatorname{sign}(x_j)\, x_j = |x_j|$, gives

$$\Delta y_{i,\text{own}} = -\eta \operatorname{sign}(\delta_i) \sum_{j=1}^{d_{\text{in}}} |x_j|. \tag{25}$$

Every term of the sum has the sign of δ_i, so the terms do not cancel: the sum is d_{in} times the average of $|x_j|$. Only the sign of δ_i is left; its size has disappeared.

Step 3: solve for the learning-rate factor. Equation (25) gives one coordinate at a time; we measure the whole vector $\Delta\boldsymbol{y}_{\text{own}}$ by its RMS over i. If no δ_i is zero, then $\operatorname{sign}(\delta_i) = \pm 1$, so every coordinate has the same absolute value $\eta \sum_j |x_j|$ and only its sign changes with i. Squaring removes the sign, so by Eq. (8) the RMS equals this common value: $\operatorname{rms}(\Delta\boldsymbol{y}_{\text{own}}) = \eta \sum_j |x_j|$. With the coordinates of $\boldsymbol{x}$ of fixed typical size, the average of $|x_j|$ does not change with width, and the change scales as $\eta\, d_{\text{in}}$. Let η_0 belong to the base model, with fan-in ratio $r_{\text{in}} = d_{\text{in}}/d_{\text{in},0}$. Keeping the change at its base value requires

$$\boxed{\frac{\eta}{\eta_0} = \frac{1}{r_{\text{in}}}.} \tag{26}$$

In this calculation, the magnitude of $\boldsymbol{\delta}$ cancels in the normalization. A hidden projection, with $r_{\text{in}} = r$, therefore needs the factor $1/r$.

Actual Adam uses the first moment divided by the square root of the second moment, not simply the sign of the current gradient. A common positive factor on the gradient history scales both quantities equally, so it cancels when ϵ is negligible or scaled along with the gradients. With batches and running averages, the exact sign formula above no longer holds. The width rule survives in the μP analysis because normalized updates remain of order one and correlated with the training inputs [18, Appendix J.1.2–J.1.3].

3.3 Forward scaling: why the readout needs $a = 1/r$

Section 3.1 solved the starting-point problem: with a controlled input RMS, fan-in initialization keeps a layer's initial output scale stable as width changes. Section 3.2 controlled the change a layer makes through its own update. Its input also changes, because earlier layers learn. We therefore need to understand how a layer transforms the changes arriving from earlier layers, as well as the initial features themselves.

Take the *readout*, the final map from hidden features to prediction scores called logits. Suppose we initialize its weights with the fan-in rule and use no extra forward scaling, so $a = 1$. Its initial logits are well scaled. Now let the earlier layers learn: they change the readout's input by the same RMS amount in a narrow model and a wide model. We will see that, for a change correlated with the readout weights, the wider model can produce a larger logit change even though both models started well scaled. Following this example will show what initialization leaves uncontrolled and how a forward multiplier fixes it.

Step 1: identify what fan-in controls and what remains to control. Let $\boldsymbol{h} \in \mathbb{R}^d$ be one token's final hidden vector, $W_{\text{out}} \in \mathbb{R}^{V\times d}$ the readout matrix, and $\boldsymbol{z} \in \mathbb{R}^V$ the logits. The vocabulary size V stays fixed as hidden width d grows; the readout here does not share weights with the input embedding. Write W_{ij} for its entries and $\boldsymbol{w}_i$ for row i, viewed as a vector:

$$z_i = a\sum_{j=1}^{d} W_{ij}h_j = a\boldsymbol{w}_i^\top \boldsymbol{h}. \tag{27}$$

Use a superscript 0 for the initialized state: $\boldsymbol{h}^0$ is the initial hidden vector, z_i^0 the initial logit, and W_{ij}^0 the initial readout entries. These entries have mean zero and standard deviation σ_{out}, independently of $\boldsymbol{h}^0$. The calculation in Section 3.1, now including a, gives

$$\mathbb{E}_W[(z_i^0)^2 \mid \boldsymbol{h}^0] = a^2 d\sigma_{\text{out}}^2\,\text{rms}(\boldsymbol{h}^0)^2. \tag{28}$$

The expectation is over the initial readout weights with $\boldsymbol{h}^0$ held fixed. Thus $a = 1$ and $\sigma_{\text{out}} \propto 1/\sqrt{d}$ keep the initial logit scale fixed.

After the first training step, write W_{ij}, h_j, and z_i for the updated weights, hidden features, and logits. Both the readout and its input can change. Define $\Delta W_{ij} := W_{ij} - W_{ij}^0$ and $\Delta h_j := h_j - h_j^0$. Subtracting the initial output gives

$$\begin{aligned}
\Delta z_i &:= z_i - z_i^0 \\
&= a\sum_{j=1}^{d} W_{ij}h_j - a\sum_{j=1}^{d} W_{ij}^0 h_j^0 \\
&= a\sum_{j=1}^{d} (W_{ij}^0 + \Delta W_{ij})h_j - a\sum_{j=1}^{d} W_{ij}^0 h_j^0 \\
&= a\sum_{j=1}^{d} W_{ij}^0 (h_j - h_j^0) + a\sum_{j=1}^{d} \Delta W_{ij}h_j \\
&= \underbrace{a\sum_{j=1}^{d} W_{ij}^0 \Delta h_j}_{\text{feature-change contribution}} + \underbrace{a\sum_{j=1}^{d} \Delta W_{ij}h_j}_{\text{weight-change contribution}}.
\end{aligned} \tag{29}$$

The second term evaluates the weight change on the updated input $\boldsymbol{h}$; it includes the interaction $a\Delta W\Delta\boldsymbol{h}$. To isolate only the readout's own update as in Section 3.2, hold the input fixed. The

multiplier a scales either contribution. Whether the learning rate needs a further width factor is settled in Section 3.4. Here we study the feature-change contribution, denoted $\Delta z_{i,\text{feature}} := a\sum_j W^0_{ij}\Delta h_j$. Its size depends on both the RMS of $\Delta \boldsymbol{h}$ and its direction relative to the initial readout weights.

Step 2: explain why training can relate the features to the weights. Let $\mathcal{L}$ be the loss and $\delta_k := \partial\mathcal{L}/\partial z_k$ its derivative with respect to logit k. Since $\partial z_k/\partial h_j = aW_{kj}$, the chain rule gives

$$\frac{\partial \mathcal{L}}{\partial h_j} = \sum_{k=1}^{V} \frac{\partial \mathcal{L}}{\partial z_k}\frac{\partial z_k}{\partial h_j} = a\sum_{k=1}^{V} \delta_k W_{kj}. \tag{30}$$

For the first update, this gradient is evaluated at initialization, where the readout entries are W^0_{kj}. The initial readout weights therefore enter the signal used to train the earlier layers. Their later features need not remain independent of these weights. The actual $\Delta \boldsymbol{h}$ also depends on those layers and the optimizer; it is not simply this gradient multiplied by a learning rate. To see the consequence of alignment without making that assumption, we next isolate a change along one readout row.

Step 3: calculate an aligned change with a fixed RMS. Write $\boldsymbol{w}^0_i$ for row i of the initial readout, and define its RMS as s_i:

$$s_i := \text{rms}(\boldsymbol{w}^0_i) = \sqrt{\frac{1}{d}\sum_{j=1}^{d}(W^0_{ij})^2}, \qquad \sum_{j=1}^{d}(W^0_{ij})^2 = ds_i^2. \tag{31}$$

For a nonzero row, take a change in its negative direction, divided by s_i and multiplied by a width-independent number $b > 0$:

$$\Delta h_j = -b\frac{W^0_{ij}}{s_i}, \qquad \text{rms}(\Delta \boldsymbol{h}) = \frac{b}{s_i}\,\text{rms}(\boldsymbol{w}^0_i) = b. \tag{32}$$

Here b is the feature-change RMS, not a learning rate. The division by s_i makes that RMS exactly the same at every width. Substitute this change into the first term of Eq. (29):

$$\begin{aligned} \Delta z_{i,\text{feature}} &= a\sum_{j=1}^{d} W^0_{ij}\left(-b\frac{W^0_{ij}}{s_i}\right) \\ &= -\frac{ab}{s_i}\sum_{j=1}^{d}(W^0_{ij})^2 \\ &= -\frac{ab}{s_i}\,ds_i^2 = -abd\,s_i. \end{aligned} \tag{33}$$

Every product has become a negative square, so the terms add with the same sign, as in Eq. (25). This is the key difference from the independent random products in Section 3.1. For a wide, independently initialized row, the average of its squared entries is close to their expectation σ^2_{out}, so $s_i \approx \sigma_{\text{out}}$. Hence

$$|\Delta z_{i,\text{feature}}| \approx ab\,d\sigma_{\text{out}}. \quad \text{With } a = 1,\ \sigma_{\text{out}} \propto d^{-1/2}: \quad |\Delta z_{i,\text{feature}}| \propto b\sqrt{d}. \tag{34}$$

The feature-change RMS stayed at b, but this contribution to the logit change grew as $\sqrt{d}$. Since V is fixed, the RMS of this contribution to the logit vector also grows. This calculation concerns one contribution, not the total change including the readout's own update.

This calculation does not claim that every real training step aligns perfectly with one row. It shows why fan-in alone cannot guarantee both nonvanishing feature changes and bounded logits. The general μP analysis accounts for training-induced correlations and obtains the same readout width rule [18, Appendices J.1.4 and J.2.1].

Step 4: solve for the forward multiplier. Equation (34) depends on the product $a\sigma_{\text{out}}$. To make a fixed-RMS aligned feature change produce a logit change of the same width order, choose

$$a\, d\sigma_{\text{out}} = \text{a width-independent constant} \quad \Longrightarrow \quad \boxed{a\sigma_{\text{out}} \propto \frac{1}{d}.} \tag{35}$$

The effective weights used in the forward pass are aW_{out}, so this rule asks their entries to shrink as $1/d$. Let d_0 be the base width, $r = d/d_0$, and $\sigma_{\text{out},0}$ the base readout standard deviation, with base multiplier one. This note keeps the stored readout at its base standard deviation and puts the whole factor into the multiplier:

$$\boxed{\sigma_{\text{out}} = \sigma_{\text{out},0}, \qquad a = \frac{1}{r} = \frac{d_0}{d}.} \qquad z_i = d_0 \left(\frac{1}{d} \sum_{j=1}^{d} W_{ij} h_j \right). \tag{36}$$

The widening sum is now an average, times the fixed number d_0. Other splits between σ_{out} and a are possible, for instance fan-in initialization with $a = 1/\sqrt{r}$, but each split comes with its own learning-rate factor (Section 3.4).

With this choice, the initial logits become smaller as width grows. For base and target hidden inputs with the same RMS, Eq. (28) gives

$$\frac{\sqrt{\mathbb{E}_W[(z^0_{i,\text{target}})^2 \mid \boldsymbol{h}^0_{\text{target}}]}}{\sqrt{\mathbb{E}_W[(z^0_{i,\text{base}})^2 \mid \boldsymbol{h}^0_{\text{base}}]}} = \frac{1}{r}\sqrt{\frac{d}{d_0}} = \frac{1}{\sqrt{r}}. \tag{37}$$

μP permits this: the logits must remain bounded, whereas hidden features must retain a nonvanishing scale. Initial logits may tend to zero; training can still produce finite, nonzero outputs [18, Appendix J.2].

Why the input layer keeps its base initialization and $a = 1$. An input embedding selects one row of a table. Increasing hidden width adds coordinates to that row; each output coordinate still selects a single stored value, so there is no widening sum to compensate for. A linear input projection with a fixed raw-input dimension has the same property: its fan-in stays fixed as hidden width grows. These input layers still need an initialization, but its scale can be copied from the base model, and no width-dependent forward multiplier is needed.

Why a hidden projection keeps fan-in initialization and $a = 1$. A hidden projection has both input and output dimensions growing with width. For its Gaussian fan-in-initialized matrix, the maximum amplification of a vector's L^2 length stays bounded with high probability when the input-to-output width ratio is fixed, even for correlated inputs [18, Appendix J.1.5]. To convert these lengths to RMS, we divide by the square roots of the respective dimensions (Eq. (9)); because both dimensions grow together, this conversion introduces no growing factor in the RMS ratio. A readout has a fixed output dimension, so it lacks this cancellation. The random initial hidden matrix can therefore keep $a = 1$.

3.4 How initialization, forward multipliers, and learning rates fit together

These three choices are not three copies of the same correction. If we change one source at a time, they appear in different parts of the computation:

$$\begin{aligned} \text{initial output:} \quad & \boldsymbol{y}_0 = aW_0\boldsymbol{x}_0, \\ \text{input-only change:} \quad & \Delta\boldsymbol{y}_{\text{input}} = aW\Delta\boldsymbol{x}, \\ \text{one weight update:} \quad & \Delta\boldsymbol{y}_{\text{own}} = -a\eta U\boldsymbol{x}. \end{aligned} \tag{38}$$

Initialization directly chooses W_0 and therefore the starting computation; it is not reapplied to later updates. The forward multiplier a acts every time the layer runs, so it affects all three lines. The learning rate η affects only the feature change created by this layer's own weight update.

For the table below, r is the target-to-base fan-in ratio of the relevant matrix.

Parameter role	Stored init.	Forward a/a_0	AdamW learning-rate factor
Input embedding	copy base	1	1
Hidden projection	std. $\times r^{-1/2}$	1	$1/r$
Stored readout	copy base	$1/r$	1
Hidden norm gain or bias	copy base	1	1

Hidden projections use initialization and learning-rate scaling. At initialization, each hidden output sums r times as many independent zero-mean products as in the base model; reducing the stored weight standard deviation by $r^{-1/2}$ cancels the resulting $\sqrt{r}$ growth. During training, its own update is correlated with its input, and Eq. (26) gives the learning rate that keeps the resulting change fixed: the factor $1/r$. Fan-in initialization controls the random initial matrix acting on a changing input (Section 3.3); the learning-rate rule controls the learned weight changes. Together these choices allow the hidden projection to keep $a = 1$.

The readout puts its width factor in the forward pass. Section 3.3 showed that the effective readout weight aW_{out} needs entries of order $1/d$, which shrink as $1/r$ rather than as the $r^{-1/2}$ of a hidden projection. This note supplies the whole factor through $a = 1/r$ and copies the base scale for the stored W_{out}. The multiplier also scales the readout's own update, $\Delta z_{k,\text{own}} = -a\eta \sum_j U_{kj}h_j$. As in Eq. (25), the d terms of this sum have size of order one and the same sign, so the change scales as $a\eta\, d \propto \eta$. The readout therefore needs no learning-rate factor: the multiplier already supplies the $1/r$ that Eq. (26) would put on the learning rate, and applying both would count it twice.

The split between stored initialization and a forward multiplier is a convention, provided the optimizer settings move with it. To see the equivalence, compare two Adam runs on the same batches, with matched initial weights $W' = W/\theta$, zero initial moment buffers, no gradient clipping, and $\epsilon = 0$. For any $\theta > 0$, the change

$$a' = a\theta, \qquad \sigma'_W = \sigma_W/\theta, \qquad \eta' = \eta/\theta \tag{39}$$

preserves the network function at every step [18, Lemma J.1]. Matching the initial draws, rather than merely their distributions, makes this a step-by-step equality. For AdamW with the decay convention $W \leftarrow (1 - \eta\lambda)W$, the shrinkage must also match:

$$\eta'\lambda' = \eta\lambda, \qquad \eta' = \eta/\theta \quad \Longrightarrow \quad \lambda' = \theta\lambda. \tag{40}$$

Here λ is the weight-decay coefficient. If a nonzero Adam ϵ is retained, it must also become $\epsilon' = \theta\epsilon$, because the gradient with respect to W' is θ times larger. Moving only the forward multiplier does not preserve training dynamics.

Lookup and vector parameters have no growing fan-in sum. An input embedding selects a row, while a normalization gain or bias acts one coordinate at a time. They therefore copy their chosen base initialization and use no width-dependent forward multiplier. A fixed semantic initialization, such as a normalization gain of one, a bias of zero, or a zero-initialized gate, remains exactly that value at every width. Their updates have no growing sum either, so Eq. (26) with $r_{\text{in}} = 1$ keeps their base learning rates.

Forward scaling inside attention. Let $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{d_{\text{head}}}$ be a query and a key with width-independent RMS. Independent, zero-mean initial coordinates give a dot product of typical size $\sqrt{d_{\text{head}}}$. During training, correlation can make the average product $q_j k_j$ nonzero and independent of width; the sum then grows as d_{head}. The original Transformer μP therefore uses an attention factor proportional to $1/d_{\text{head}}$ [18, Definition 4.1]:

$$\text{attention score} \;\propto\; \frac{\boldsymbol{q}^\top \boldsymbol{k}}{d_{\text{head}}} = \frac{1}{d_{\text{head}}} \sum_{j=1}^{d_{\text{head}}} q_j k_j. \tag{41}$$

If width grows by adding fixed-size heads, d_{head} and this factor stay fixed.

RMSNorm and the remaining Transformer operations. RMSNorm's gain is usually initialized to one, independently of width; RMSNorm usually has no bias. LayerNorm likewise normally starts with gain one and bias zero. These are vector parameters, so the table's vector rule applies [18, Table 3]. The normalization stabilizer and Adam's optimizer stabilizer are different settings; the ϵ scaling discussed below concerns only Adam.

Softmax, SiLU or GELU, the SwiGLU elementwise product, residual addition, and RoPE have no trainable tensors of their own, so they do not need separate initialization or learning-rate rules.

4 CompleteP: Depth Needs More than Stability

4.1 What should stay stable when we add more layers?

Section 3 controlled width. Now hold width fixed and increase depth, keeping the input scale, training duration, and base hyperparameters fixed. We want two quantities to stay at comparable scales: the RMS of the hidden features at initialization, and the RMS of their change after a training step. Neither should blow up; neither should vanish merely because the network is deeper. "Comparable" means the same order of magnitude, not exactly the same number. In particular, we care about $\text{rms}(\Delta \boldsymbol{h})$, not the difference $\text{rms}(\boldsymbol{h} + \Delta \boldsymbol{h}) - \text{rms}(\boldsymbol{h})$.

Start with the simplest residual network. A residual branch adds a correction to the current hidden state. To isolate what repeated addition does, use one scalar coordinate and let each branch add one trainable number w_ℓ:

$$h_0 = 1, \qquad h_{\ell+1} = h_\ell + w_\ell \quad (\ell = 0, \dots, L-1), \qquad h_L = 1 + \sum_{\ell=0}^{L-1} w_\ell. \tag{42}$$

Here L counts additions, ℓ identifies a branch, and h_L is the final hidden state. This deliberately simple branch ignores its input; it lets us calculate accumulation exactly before considering an MLP or attention.

Initialize the weights w_ℓ^0 independently with mean zero and variance one. The superscript 0 denotes initialization. Since independent variances add, while the constant input contributes no variance,

$$\begin{aligned} \mathbb{E}[h_L^0] = 1, \qquad \mathrm{Var}(h_L^0) = \sum_{\ell=0}^{L-1} \mathrm{Var}(w_\ell^0) = L, \\ \mathbb{E}[(h_L^0)^2] = \mathrm{Var}(h_L^0) + (\mathbb{E}[h_L^0])^2 = L + 1. \end{aligned} \tag{43}$$

For this scalar model, $\sqrt{\mathbb{E}[(h_L^0)^2]}$ is the root mean square over random initializations. It grows as $\sqrt{L+1}$: each branch started with a fixed scale, but adding more branches made the hidden state larger.

Introduce a coefficient only now. Multiply each correction by an as-yet-undetermined positive number c_L:

$$h_{\ell+1} = h_\ell + c_L w_\ell, \qquad h_L = 1 + c_L \sum_{\ell=0}^{L-1} w_\ell, \qquad \mathbb{E}[(h_L^0)^2] = 1 + L c_L^2. \tag{44}$$

The square on c_L comes from variance: multiplying a random number by c_L multiplies its variance by c_L^2. Choosing $c_L = 1/\sqrt{L}$ makes $Lc_L^2 = 1$. Choosing $c_L = 1/L$ also prevents growth, giving $Lc_L^2 = 1/L$. In the latter case, the random corrections shrink, but the whole hidden state retains the input $h_0 = 1$. Thus initialization stability alone cannot tell us which of these two choices to use.

Training adds another constraint. Let $\Delta w_\ell := w_\ell - w_\ell^0$ be the actual change to a branch parameter in one training step. Unlike the independent initial weights, these changes can point in the same direction: all branches are trained against the same output error. Indeed, for any loss $\mathcal{L}(h_L)$, $\partial\mathcal{L}/\partial w_\ell = c_L\, \partial\mathcal{L}/\partial h_L$ is the same for every branch. Consider equal updates $\Delta w_\ell = \delta_L$ and write $u_L := |\delta_L|$ for their magnitude. Then

$$\Delta h_L := h_L - h_L^0 = c_L \sum_{\ell=0}^{L-1} \Delta w_\ell = L c_L \delta_L, \qquad |\Delta h_L| = L c_L u_L. \tag{45}$$

There is no square root here: aligned updates add directly. With no scaling and a fixed update magnitude, the feature change grows as L. To keep it at a nonzero, depth-independent size, we instead need

$$L c_L u_L = \text{a positive constant}, \qquad u_L \propto \frac{1}{L c_L}. \tag{46}$$

Thus $c_L = 1/\sqrt{L}$ needs $u_L \propto 1/\sqrt{L}$, whereas $c_L = 1/L$ allows u_L to stay independent of depth. Both control the total update, but they permit different amounts of learning inside each branch. These are actual update magnitudes; the learning rate needed to realize them depends on the optimizer.

The exponent summarizes these choices. We have now encountered 1, $1/\sqrt{L}$, and $1/L$. Writing $c_L \propto L^{-\alpha}$ puts them in one family, with $\alpha = 0, 1/2, 1$ respectively. A power law makes the rule depend only on the depth ratio: doubling depth always multiplies the coefficient by $2^{-\alpha}$. It is a useful family of rules to analyze, not a claim that every possible stable coefficient must be an exact power law.

For a base depth L_0, define $q := L/L_0$. Absorb the base coefficient into the definition of the branch F_ℓ; its additional multiplier is then $c_L/c_{L_0} = q^{-\alpha}$. Restoring a vector-valued branch gives

$$\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + q^{-\alpha} F_\ell(\boldsymbol{h}_\ell), \qquad q = \frac{L}{L_0}. \tag{47}$$

The coefficient depends on total depth, not the branch position ℓ. Our two toy calculations now give the depth factors

$$\frac{u_L}{u_{L_0}} = \frac{L_0 c_{L_0}}{L c_L} = q^{\alpha-1}, \qquad \frac{L c_L^2}{L_0 c_{L_0}^2} = q^{1-2\alpha}. \tag{48}$$

The second factor must not grow, requiring $\alpha \geq 1/2$. Requiring the internal update magnitude not to grow requires $\alpha \leq 1$. This recovers the family $1/2 \leq \alpha \leq 1$ considered by CompleteP [5, Section 6]. The toy model explains the scaling logic; actual Transformer branches need not be independent at initialization or have perfectly aligned updates during training.

4.2 A moving feature can still lose nonlinear learning

The additive toy branch above has only a linear dependence on its parameter: its output change is exactly Δw_ℓ. It cannot show whether deeper networks lose nonlinear learning inside a branch. For that question, we need a branch containing a composition of trainable operations.

An MLP, for example, first applies one projection and then another, with an activation in between. The smallest model of the two trainable projections uses scalar weights w_1, w_2. Drop the activation just for this calculation:

$$x \xrightarrow{\times w_1} w_1 x \xrightarrow{\times w_2} F(w_1, w_2; x) = w_2 w_1 x. \tag{49}$$

Here x is the input held fixed while this branch learns; w_1, w_2 are weights, not two extra scaling coefficients. The product is linear in x but not jointly linear in the two weights. Write their initial values as w_1^0, w_2^0 and their updates as $\Delta w_1, \Delta w_2$. Expanding the new output and subtracting the initial one gives

$$\begin{aligned} \Delta F &= (w_2^0 + \Delta w_2)(w_1^0 + \Delta w_1)x - w_2^0 w_1^0 x \\ &= \underbrace{(w_2^0 \Delta w_1 + w_1^0 \Delta w_2)x}_{\text{first-order change}} + \underbrace{\Delta w_2 \Delta w_1 x}_{\text{interaction between updates}}. \end{aligned} \tag{50}$$

The first term treats the two weight changes separately, adding the effect of each. This is the *first-order*, or linear, prediction. The second term is their interaction: changing both weights has an extra effect that changing either one alone cannot produce. If updates shrink, the interaction can vanish relative to the first-order change even while the network as a whole continues to move.

Keep x and the initial weight scales independent of depth. Let both updates have characteristic magnitude u_L. The first-order term contains one update, so its magnitude scales as u_L; the interaction contains two, so it scales as u_L^2. Multiplying the branch by c_L multiplies both terms equally. Their relative magnitude therefore scales as

$$\frac{\text{interaction magnitude}}{\text{first-order magnitude}} \propto \frac{c_L u_L^2}{c_L u_L} = u_L \propto q^{\alpha-1}. \tag{51}$$

Here $\propto$ means proportional to: we keep the dependence on depth and omit factors that do not change with depth. For $\alpha = 1/2$, each internal update shrinks as $1/\sqrt{q}$, and so does this ratio. For $\alpha = 1$, neither shrinks merely because depth grows. An update can still be small because the chosen base learning rate is small; the requirement is that adding layers must not force it to zero. This comparison assumes that the inputs, weights, and update directions produce nonzero terms rather than accidental cancellations.

The shrinking interaction explains the concern. The whole network can change while learning inside an individual block becomes increasingly well described by its first-order prediction. This

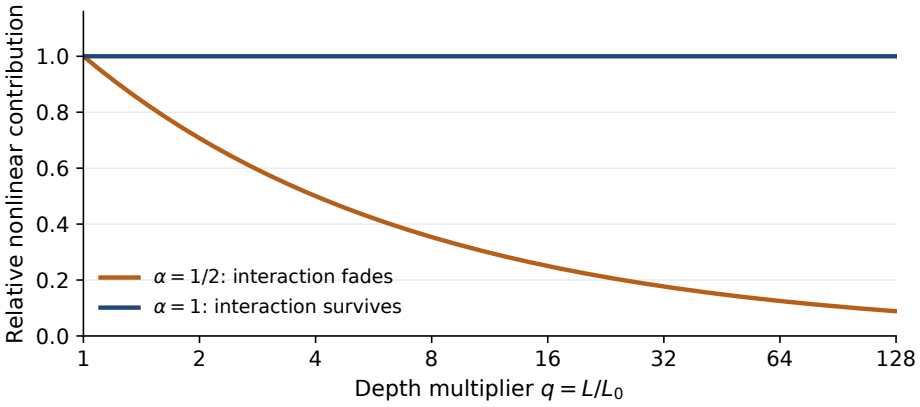


Figure 2: The depth dependence in Eq. (51), normalized to one at the base depth. This is a schematic of the toy calculation, not measured Transformer loss or a learning-rate sweep.

is called *lazy learning with respect to those parameters*. It does not mean that the model is linear in its input or unable to reduce loss.

Complete feature learning asks that such nonlinear interactions do not vanish relative to the first-order change wherever the finite network has them, including for subsets of trainable parameters. The paper extends the argument beyond two weights; within its analyzed stable depth family, $\alpha = 1$ retains the higher-order interactions [5, Section 6]. This is a stronger condition than keeping activation RMS and total feature movement well scaled.

4.3 The resulting rule, and its boundary

CompleteP combines width control with the depth choice $\alpha = 1$:

$$\boxed{\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + \frac{L_0}{L} F_\ell(\boldsymbol{h}_\ell).} \tag{52}$$

Here F_ℓ is an attention or FFN branch, including its input normalization. In the toy model, L counted residual additions. For a Transformer, return to the convention that L counts Transformer blocks: each contains two additions, one for attention and one for the FFN. Their count ratio is still $2L/(2L_0) = q$, so both branches receive the same $1/q$ multiplier. The unchanged skip connection carries $\boldsymbol{h}_\ell$ directly. The proxy has multiplier one. At twice the depth, each repeated residual branch is multiplied by $1/2$. Equation (48) now gives depth factor 1 for unscaled branch parameter updates: the updates need not shrink merely because the network is deeper. Each branch's leading contribution still receives the forward factor q^{-1}, while nonlinear interactions inside it can survive.

The important combination is therefore *persistent* $1/q$ *forward scaling plus nonshrinking unscaled branch updates*. The numerical LR needed to produce those updates depends on the optimizer. These requirements explain the depth mechanism; they do not extend an optimizer-specific theorem to arbitrary update algorithms.

At large depth, $1/L$ can make the aggregate random residual contribution small at initialization. The main hidden vector, called the *residual stream*, can remain constant order because it also carries the input representation. CompleteP controls learned changes without requiring random branch contributions to remain nonzero.

The residual multiplier persists in every forward pass. An initialization-only rule such as $\mathrm{Std}(W_{\text{writeback}}) \propto 1/\sqrt{2L}$ changes the starting weights and is not the same operation. Adding

such a rule, a post-projection norm, or a trainable gate changes the recipe and requires verification. A complete training recipe must also specify parameter groups and how their update scales are realized; Section 4.4 and Part IV cover those choices.

The evidence is strongest for the pre-LN Transformer training settings studied in the paper. Its experiments reached 1.5B non-embedding parameters and included depth transfer from 2 to 128 layers. The theoretical scaling analysis holds training tokens fixed; the authors also tested a fixed 20-tokens-per-parameter regime. These results motivate transfer, but do not guarantee an unchanged optimal LR for every architecture, batch, dataset, training horizon, or optimizer [5]; Section 5 treats the horizon and the batch.

RMSNorm under CompleteP. The CompleteP paper analyzes a pre-LayerNorm Transformer, not RMSNorm directly. It makes no width- or depth-dependent change to normalization initialization; its reference implementation initializes the gain to one and the optional bias to zero. Applying the same vector-parameter role to RMSNorm therefore keeps $\boldsymbol{\gamma}_0 = \mathbf{1}$ as width and depth change. The $1/q$ depth factor multiplies the residual branch output, not the RMSNorm initialization [5, Table 1 and Appendix D.3] [4].

4.4 Learning rates across depth, ϵ, and weight decay

CompleteP fixes the forward pass. The learning rates must still keep the unscaled branch updates at their base size (Eq. (48) with $\alpha = 1$). The derivation of Section 3.2 applies to a matrix inside a branch, whose output lies before the branch factor $1/q$. That factor also multiplies the backward signal entering the branch [15, Section 3.3], so every gradient inside a branch carries an extra $1/q$. In AdamW's normalized update, a common positive gradient factor multiplies both the first moment and the square root of the second moment, so it cancels when ϵ is negligible or scaled with the gradients. Thus this factor does not itself shrink the update; the learning rate sets its overall magnitude. With $\alpha = 1$ the required update factor is one, and AdamW's learning rate needs no depth factor [5, Table 1 and Appendix D.3]. In the general family of Eq. (47), branch parameters, including hidden pre-normalization gains, take $q^{\alpha-1}$.

Adam's stabilizer ϵ. Adam divides by $\sqrt{\widehat{v}} + \epsilon$, where $\sqrt{\widehat{v}}$ tracks the size of the gradient, and that size shrinks with width. At the logits, $\boldsymbol{\delta}$ does not depend on width, because the vocabulary is fixed. Equation (30) passes it through the readout, multiplying by $a = 1/r$ and summing over the fixed V logits, so the signal reaching the last hidden vector scales as $1/r$. With hidden width ratios held fixed, hidden projections, residual additions, and RMSNorm preserve this order [15, Section 3.1]. Each gradient entry is such a δ times a width-independent number: $\delta_i x_j$ for a hidden matrix, δ_j for an embedding row, $\delta_i \hat{x}_i$ for a gain acting on the normalized coordinate $\hat{x}_i$, and δ_i for a hidden bias. In the readout's gradient $a\delta_k h_j$, the logit derivative δ_k is of order one and $a = 1/r$ supplies the same factor. These parameter groups therefore have gradient coordinate scales of order $1/r$ relative to the base, and $1/(rq)$ inside branches. This is a scaling statement under the declared family, not equality of individual gradients between two independently trained models.

A fixed ϵ does not shrink, so a stabilizer that is negligible in the base model can dominate in a larger one. Adam's step then becomes approximately $-(\eta/\epsilon)\widehat{m}$, proportional to the gradient rather than normalized by its second moment, so the update shrinks. Keeping ϵ at a fixed fraction of $\sqrt{\widehat{v}}$ requires [15, Appendix B][5, Table 1]

$$\frac{\epsilon}{\epsilon_0} = \frac{1}{r}, \qquad \text{and } \frac{1}{rq} \text{ inside residual branches.} \tag{53}$$

A bias added *after* the readout multiplier is an exception: for $\boldsymbol{z} = r^{-1}W\boldsymbol{h} + \boldsymbol{b}_{\text{out}}$, $\partial\mathcal{L}/\partial\boldsymbol{b}_{\text{out}} = \boldsymbol{\delta}$ has no $1/r$ factor. Its Adam ϵ therefore stays at its base scale. Recheck any extra shared tensor from its actual computation graph. Simpler alternatives for the listed groups are an ϵ far below $\sqrt{\widehat{v}}$ at the largest model, or Adam-atan2, which has no ϵ [6]. Either way, log $\sqrt{\widehat{v}}/\epsilon$ for each group.

Weight decay. In the common implementation, AdamW multiplies each weight by $1 - \eta\lambda$ per update, where λ is the decay coefficient and η the group's learning rate. Holding the input fixed, this changes the layer's output by the same fraction:

$$W \leftarrow (1 - \eta\lambda)W \quad \Longrightarrow \quad \Delta\boldsymbol{y}_{\text{decay}} = -\eta\lambda\,\boldsymbol{y}. \tag{54}$$

This fraction has no width factor of its own, so width transfer alone keeps $\eta\lambda$ at its base value: $\lambda/\lambda_0 = \eta_0/\eta$ in every group. This gives $\lambda \propto r$ on hidden matrices [13][15, Section 3.4]. Under CompleteP, λ needs no depth correction [5, Table 1 and Appendix D.4]. An implementation that applies the decay without the learning rate needs no factor.

Decay also accumulates over the run. For group learning rates η_t, its gradient-free shrinkage is

$$\prod_{t=0}^{S-1}(1 - \eta_t\lambda) \approx \exp\left(-\lambda\sum_{t=0}^{S-1}\eta_t\right). \tag{55}$$

Thus copying λ does not preserve the total decay when the horizon, batch, or schedule changes. For a constant rate, the decay timescale in units of the full training run is $1/(\eta\lambda S)$; for a schedule, use the LR sum above. Keeping that timescale fixed is a useful candidate rule, not a universal optimum [17].

In their compute-optimal experiments, Qiu et al. found that a per-update shrinkage coefficient decreasing inversely with width worked well for the tested optimizers [15, Section 3.4]. Under the hidden-matrix AdamW rule $\eta \propto 1/r$, this corresponds to a constant λ if no extra horizon or batch correction changes η. It is an empirical joint-scaling recipe; it does not replace the fixed-horizon rule above. Choose which comparison is intended before copying decay.

5 Learning Rate beyond Width and Depth

Sections 3 and 4 move the learning rate across width and depth while the training length, batch, and schedule stay fixed; the derivations behind μP assume such a fixed horizon [6, 2]. A scaling ladder does not keep them fixed. Under compute-optimal allocation (Part I, Section 4), a larger model also trains on more tokens, often with a larger batch, and its schedule must end at the new length. This section asks how the best learning rate depends on training length, batch, and schedule, and what to fit when μP is not used. Here log is the natural logarithm.

5.1 One learning rate, two effects

A model with a single weight w shows the mechanism. Let its loss be $\mathcal{L}(w) = \frac{H}{2}w^2$, with curvature $H > 0$ and minimum at $w = 0$. Step t sees the noisy gradient $Hw_t + \zeta_t$, where the noise ζ_t has mean zero and variance σ^2 and is independent of w_t. SGD with learning rate η_t gives

$$w_{t+1} = w_t - \eta_t(Hw_t + \zeta_t) = (1 - \eta_t H)\,w_t - \eta_t\zeta_t. \tag{56}$$

Square both sides and average. The cross term averages to zero by independence, as in Eq. (5), so the second moment $m_t := \mathbb{E}[w_t^2]$ obeys

$$m_{t+1} = (1 - \eta_t H)^2\,m_t + \eta_t^2\sigma^2. \tag{57}$$

For a constant η with $0 < \eta H < 2$, the fixed point $m_\infty = \eta^2\sigma^2/[1-(1-\eta H)^2]$ satisfies the same recursion, so the gap $m_t - m_\infty$ shrinks by the factor $(1-\eta H)^2$ every step. With $\mathbb{E}[\mathcal{L}_t] = \frac{H}{2} m_t$ and $\mathcal{L}_0 := \frac{H}{2} m_0$, the expected loss after S steps is

$$\mathbb{E}[\mathcal{L}_S] = \underbrace{(1-\eta H)^{2S}\,(\mathcal{L}_0 - \mathcal{L}_{\text{noise}})}_{\text{progress}} + \underbrace{\mathcal{L}_{\text{noise}}}_{\text{noise floor}} \qquad \text{with} \qquad \mathcal{L}_{\text{noise}} = \frac{\eta\sigma^2}{2(2-\eta H)}. \tag{58}$$

The two terms respond differently to η. When $\eta H \ll 1$, $\log(1-\eta H) \approx -\eta H$, so the progress factor is about $e^{-2H\eta S}$. For a varying rate, the same approximation applied to Eq. (57) gives the factor

$$\prod_{t=0}^{S-1} (1-\eta_t H)^2 \approx e^{-2H\Lambda(S)}, \qquad \Lambda(S) := \sum_{t=0}^{S-1} \eta_t. \tag{59}$$

In this approximation, contraction of the initial error depends on the rates through Λ, the area under the learning-rate curve. For a constant rate the noise floor is about $\eta\sigma^2/4$. Under a varying schedule, accumulated noise depends on the whole rate history; it does not instantly reach the floor of the current rate. A larger rate makes faster progress but settles higher. This is the classical picture of SGD with a fixed step, whose expected excess loss approaches a level proportional to the step [3], and of the noisy quadratic model used to study batch size [19]. A network has many curvatures, and Adam rescales each coordinate, so the model predicts the direction of each effect, not its size.

5.2 Why longer training can favor a smaller learning rate

Fix the number of steps S and choose the constant η that minimizes Eq. (58). When $\eta H \ll 1$ and the initial loss lies far above the floor,

$$\mathbb{E}[\mathcal{L}_S] \approx \mathcal{L}_0\, e^{-2H\eta S} + \frac{\eta\sigma^2}{4}. \tag{60}$$

Setting the derivative with respect to η to zero, $-2HS\mathcal{L}_0 e^{-2H\eta S} + \sigma^2/4 = 0$, gives

$$\eta^*(S) = \frac{1}{2HS} \log \frac{8HS\mathcal{L}_0}{\sigma^2}. \tag{61}$$

This interior approximation requires positive noise, a positive logarithm, and a resulting $\eta^* H \ll 1$. Once the logarithm exceeds one, increasing S decreases this candidate optimum. Up to that logarithm, the dependence is $1/S$. Boundary optima or nearly noiseless training need separate analysis. The same equation explains why the loss curves of two constant rates $\eta_1 < \eta_2$ cross. Early on, the progress term dominates and η_2 is ahead; late in training, both progress terms have vanished and each curve approaches its own floor, with $\eta_1\sigma^2/4 < \eta_2\sigma^2/4$. A rate tuned on a short pilot can therefore be too large for a long run.

Language-model measurements confirm the direction and supply the size. At fixed model size, Bjorck et al. fit

$$\frac{\eta^*(D)}{\eta^*(D_0)} = \left(\frac{D}{D_0}\right)^{-\beta_D}, \qquad \beta_D \approx 0.32, \tag{62}$$

where D is the number of training tokens and batch is held fixed, for models of 760M parameters and up in their experiments; smaller models show a larger exponent [2]. Fitted on horizons of 25B–100B tokens, the law predicted the optimum at longer horizons within 10–15% for a 50M model, against more than 250% error for an unchanged rate; the optimum does not transfer across horizons even with μP. The $1/S$ of Eq. (61) is not this exponent; only its direction carries over. Treat 0.32 as a starting prior for comparable settings. Changing the batch, decay, schedule, or data distribution requires checking the correction again.

Batch Hold the training data fixed at D examples. A batch of B examples gives $S = D/B$ steps and averages B independent per-example noises of variance σ_1^2, so $\sigma^2 = \sigma_1^2/B$ by Eq. (7) with $p_j = 1/B$. Substituting both into Eq. (60) gives

$$\mathbb{E}[\mathcal{L}_S] \approx \mathcal{L}_0 \exp\left(-2HD\,\frac{\eta}{B}\right) + \frac{\sigma_1^2}{4}\,\frac{\eta}{B}. \tag{63}$$

The rate and the batch enter only through η/B, so for SGD the best rate grows in proportion to the batch. The derivation needs $\eta H \ll 1$, which fails at large batch: McCandlish et al. show that the optimal SGD step grows linearly with B only below the gradient noise scale, their estimate of the critical batch (Part I, Section 3.4), and saturates above it [12]. For Adam, an SDE analysis gives a square-root rule instead: doubling the batch multiplies the rate by $\sqrt{2}$, with matching changes to β_1, β_2, and ϵ under the analysis's small-step and noise assumptions [11]. It is not a rule for changing LR alone at any batch. Both rules give a starting point for the batch sweep of Part IV, Section 4, not a substitute for it.

5.3 Schedules: reduce noise after early progress

Equation (57) also explains why schedules end with a decay. The noise part of m_t relaxes toward the floor of the current rate by the factor $(1 - \eta_t H)^2 \approx e^{-2H\eta_t}$ per step, that is, over about $1/(2H\eta_t)$ steps, while progress depends on Λ. A run that holds a high rate and then lowers it can reduce the initial error quickly, then reduce the contribution of accumulated noise. Shedding takes time, because relaxation at a low rate is slow, so the decay cannot be instantaneous.

Two empirical laws model the loss along a whole schedule with these two ingredients. Tissue et al. write the loss at step s as

$$\mathcal{L}(s) = \mathcal{L}_\infty + A_1\,\Lambda(s)^{-\nu} - A_2\,\Lambda_{\mathrm{dec}}(s), \tag{64}$$

where Λ is the area of Eq. (59), Λ_{dec} accumulates the decreases of the rate, smoothed by a momentum-like factor between 0.99 and 0.999, and $\mathcal{L}_\infty$, A_1, A_2, ν are fitted [16]. Luo et al.'s multi-power law lets the gain from each decrease build up and saturate over the following steps [10]. These are empirical loss predictors within their calibration regimes, not consequences of the scalar quadratic. Using them to compare schedules requires matching the training setup and validating on held-out schedules; choosing the peak rate still needs a sweep.

The decay also enables a cheaper ladder. MiniCPM's warmup–stable–decay schedule holds the rate constant after warmup and decays it only at the end [8]. Hägele et al. find that a constant rate followed by a short cooldown, at most about 20% of the steps, matches a cosine schedule of the same length in their tested settings [7]. One constant-rate run per model size can then branch into cooldowns at several lengths, each a finished run, so a scaling ladder needs fewer runs. This avoids the unfinished cosine schedules behind Kaplan's checkpoints (Part I, Section 4.1).

5.4 Without μP: fit the learning rate directly

Without μP, the best rate shifts with model size, so studies fit it. Kaplan et al. used the rule of thumb $\eta(N) \approx 0.003239 - 0.0001395 \log N$ for N non-embedding parameters and noted that it likely also depends on width through the initialization scale [9, Appendix D.6], which is the dependence μP derives. DeepSeek LLM fits, as a function of training compute C alone,

$$\eta_{\mathrm{opt}} = 0.3118\,C^{-0.1250}, \tag{65}$$

alongside its batch rule (Part I, Section 5.1), with C in its own FLOP convention [1]. Porian et al. fit the optimal rate and batch as power laws in model size at 20 tokens per parameter [14].

Each rule is measured like the allocation of Part I, Section 4.2: at every scale, sweep η on a geometric grid and fit a parabola in $\log \eta$ to locate the optimum, as Bjorck et al. do [2]; then fit a power law to the optima across scales. Such a rule merges width, horizon, and batch into one exponent and holds only for the parametrization, data, optimizer, and schedule it was measured with; DeepSeek observes that its optimum shifts even when the same compute is split differently between model and data.

With μP and CompleteP, apply the tensor and branch factors first, then measure any remaining drift in the best *base* rate. This removes known scaling factors from the fit; it does not prove that horizon, batch, and architecture effects are independent. For example, a pilot might test

$$\eta_g = \eta_{g,0} f_g(r, q)\, H(D/D_0, B/B_0), \tag{66}$$

where f_g is the group factor from Sections 3 and 4, and H is an empirically checked correction. A factorized choice $H = (D/D_0)^{-\beta_D}(B/B_0)^{\gamma}$ is a hypothesis, not an additional μP theorem. Fit horizon changes at fixed batch and batch changes at fixed horizon before testing their combination.

Everett et al. observed a stronger downward LR drift when tokens grew with model size than in fixed-step sweeps [6, Appendix I]. This is consistent with a horizon effect, but does not establish that Bjorck et al.'s exponent predicts those experiments: the protocols differ. Qiu et al. also report a regime where a transferred base rate remains near-optimal as tokens and model size grow [15, Section 3.4]. Together these results argue for a measured correction, which may be small, rather than automatically multiplying several published exponents. Part I, Section 5.3 explains how to hold this joint recipe consistent along a scaling ladder.

6 Summary: Transfer, Then Check

The base recipe fixes constants; the width and depth rules tell us how those constants are used in a larger model, and Section 5 adjusts the learning rate for batch and training length. Keep the architecture family explicit: hidden dimensions grow together, vocabulary stays fixed, and changes in head dimension, sequence length, batch, and training duration are declared.

6.1 Width and depth rules in one place

Let $r = d/d_0$ and $q = L/L_0$. The table combines μP with CompleteP, using the stored-weight convention of Section 3.4. Every entry is a target-to-base factor. Hold batch, training length, and schedule fixed for this comparison; their corrections come afterward.

Tensor or operation	Init. std.	Forward	AdamW LR
Input embedding	1	1	1
Hidden projection	$r^{-1/2}$	1	$1/r$
Stored readout	1	$1/r$	1
Hidden norm gain or bias	1	1	1
Each residual branch	—	$1/q$	—

Apply width factors to tensors, then the depth factor to each whole branch. A hidden matrix inside a branch still uses initialization factor $r^{-1/2}$ and learning-rate factor

$1/r$. CompleteP multiplies the completed branch output by $1/q$ before it is added to the skip connection:

$$\boldsymbol{h}_{\ell+1} = \boldsymbol{h}_\ell + \frac{1}{q} F_\ell(\boldsymbol{h}_\ell).$$

Do not multiply every internal projection by $1/q$: that would apply the same branch factor repeatedly. In a Transformer, both the attention branch and the FFN branch receive it.

CompleteP adds no depth factor to initialization or AdamW's learning rate. This is deliberate: the forward factor controls accumulation across layers, while the unscaled branch updates retain their size. It assumes Adam's normalization behaves as intended. For the tensor conventions derived above, its stabilizer uses $\epsilon/\epsilon_0 = 1/(rq)$ inside branches and $1/r$ for the input, stored readout, and final hidden normalization, or remains negligible at every tested scale. A bias added after the readout multiplier retains its base ϵ. For AdamW decay $W \leftarrow (1 - \eta\lambda)W$, fixed-horizon transfer keeps $\eta\lambda$ fixed within each group: the width rule gives $\lambda/\lambda_0 = r$ on hidden matrices and 1 on the other parameter rows, with no extra depth factor. Groups excluded from decay stay excluded. Changing the horizon requires the separate discussion in Section 4.4.

The vector row also covers final hidden normalization; the layerwise ϵ still depends on placement. Use each matrix's own fan-in ratio; the displayed r assumes hidden widths scale together. Fixed initializations, such as gains of one or biases of zero, stay fixed. Shared input/output embeddings require a consistent shared parameter convention. If the head dimension grows, also multiply the base attention-score factor by $d_{\text{head},0}/d_{\text{head}}$; adding fixed-size heads needs no such change.

6.2 Check before scaling

A short check asks two questions: did we implement the factors correctly, and does the small model's useful learning-rate range transfer? It does not require different models to have identical activations or losses.

1. **List every parameter group.** Record its role, whether it is inside a residual branch, and its initialization, forward multiplier, learning rate, ϵ, and decay. Include gains and biases; check that every residual addition has exactly one depth multiplier.

2. **Compare a few short runs.** Vary width at fixed depth, then depth at fixed width. Keep data, batch, training steps, and schedule matched. Use a fixed probe batch for measurements, at initialization and after the same few training steps. For a feature vector $\boldsymbol{h}^t$, record $\text{rms}(\boldsymbol{h}^t)$ and $\text{rms}(\boldsymbol{h}^t - \boldsymbol{h}^0)$, where t counts updates and 0 denotes initialization. When depths differ, compare layers at the same relative position ℓ/L.

   Check the expected scaling of each quantity, not universal flatness. Hidden features and residual-stream changes should stay on comparable scales once learning begins. In contrast, nonzero randomly initialized logits in our readout convention shrink as $r^{-1/2}$, and a scaled branch output shrinks as $1/q$ when its unscaled output has depth-independent RMS. Plot $q\ \text{rms}(F_\ell/q) = \text{rms}(F_\ell)$ to compare the latter across depths. A deliberately zero-initialized output is zero initially; judge its learned changes after it starts moving. Unexpected growth or collapse is a reason to inspect the groups, rather than proof from a single noisy run [18, 13, 5].

3. **Check the optimizer actually makes the intended updates.** Log $\sqrt{\widehat{v}}/\epsilon$ by group across the tested widths and depths. If ϵ was meant to be negligible, this ratio should be much greater than one for active coordinates; otherwise, check that the intended relative scale is preserved. Zero-gradient coordinates need separate interpretation. Log $\eta\lambda$ for decayed groups and compare it with the chosen rule in Section 4.4.

4. **Sweep the base learning rate.** At two or more widths and depths, try a few base rates, applying each group's factors before training. Compare loss against the base rate, not against a hidden matrix's scaled rate. Good ranges should substantially overlap; exact agreement of the best sampled point is unnecessary. If the target trains longer or changes batch, apply the corrections of Section 5 and recheck. For a large scale jump, use an intermediate model before committing the full budget [6, 15].

5. **Freeze the recipe and monitor the target.** Save the group settings and run conditions, launch the larger model, and watch the same measurements early in training. Passing a short check rules out some implementation errors; it does not guarantee long-run stability or an unchanged optimum under a new data distribution.

Coordinate stability alone does not establish complete feature learning: both $\alpha = 1/2$ and $\alpha = 1$ can pass a stability check. For CompleteP, also verify the persistent $1/q$ branch multiplier and the update rules of Section 6.1. Part IV assembles these checks, batch selection, and budget allocation into one training procedure.

References

[1] Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, et al. DeepSeek LLM: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954*, 2024. URL: https://arxiv.org/abs/2401.02954.

[2] Johan Bjorck, Alon Benhaim, Vishrav Chaudhary, Furu Wei, and Xia Song. Scaling optimal LR across token horizons. In *International Conference on Learning Representations*, 2025. URL: https://arxiv.org/abs/2409.19913.

[3] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018. URL: https://arxiv.org/abs/1606.04838.

[4] Nolan Dey et al. CompleteP reference implementation. nanoGPT-mup, completep branch. Corrected implementation; accessed October 2026. URL: https://github.com/EleutherAI/nanoGPT-mup/tree/completep.

[5] Nolan Dey, Bin Claire Zhang, Lorenzo Noci, Mufan Li, Blake Bordelon, Shane Bergsma, Cengiz Pehlevan, Boris Hanin, and Joel Hestness. Don't be lazy: CompleteP enables compute-efficient deep transformers. In *Advances in Neural Information Processing Systems*, volume 38, 2025. Corrected arXiv revision, January 2026. URL: https://arxiv.org/abs/2505.01618v4, doi:10.52202/085713-4597.

[6] Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, and Jeffrey Pennington. Scaling exponents across parameterizations and optimizers. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL: https://arxiv.org/abs/2407.05872.

[7] Alexander Hägele, Elie Bakouch, Atli Kosson, Loubna Ben Allal, Leandro Von Werra, and Martin Jaggi. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL: https://arxiv.org/abs/2405.18392.

[8] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, et al. MiniCPM: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024. URL: https://arxiv.org/abs/2404.06395.

[9] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL: https://arxiv.org/abs/2001.08361, doi:10.48550/arXiv.2001.08361.

[10] Kairong Luo, Haodong Wen, Shengding Hu, Zhenbo Sun, Zhiyuan Liu, Maosong Sun, Kaifeng Lyu, and Wenguang Chen. A multi-power law for loss curve prediction across learning rate schedules. In *International Conference on Learning Representations*, 2025. URL: https://arxiv.org/abs/2503.12811.

[11] Sadhika Malladi, Kaifeng Lyu, Sreyas Panigrahi, and Sanjeev Arora. On the SDEs and scaling rules for adaptive gradient algorithms. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL: https://arxiv.org/abs/2205.10287.

[12] Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018. URL: https://arxiv.org/abs/1812.06162, doi:10.48550/arXiv.1812.06162.

[13] Microsoft. Maximal update parametrization: Implementation and coordinate checks. Reference implementation. Accessed October 2026. URL: https://github.com/microsoft/mup.

[14] Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. Resolving discrepancies in compute-optimal scaling of language models. *arXiv preprint arXiv:2406.19146*, 2024. URL: https://arxiv.org/abs/2406.19146.

[15] Shikai Qiu, Zixi Chen, Hoang Phan, Qi Lei, and Andrew Gordon Wilson. Hyperparameter transfer enables consistent gains of matrix-preconditioned optimizers across scales. *arXiv preprint arXiv:2512.05620v2*, 2026. January 2026 revision. URL: https://arxiv.org/html/2512.05620v2.

[16] Howe Tissue, Venus Wang, and Lu Wang. Scaling law with learning rate annealing. *arXiv preprint arXiv:2408.11029*, 2024. URL: https://arxiv.org/abs/2408.11029.

[17] Xi Wang and Laurence Aitchison. How to set AdamW's weight decay as you scale model and dataset size. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL: https://arxiv.org/abs/2405.13698.

[18] Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs V: Tuning large neural networks via zero-shot hyperparameter transfer. arXiv:2203.03466, 2022. URL: https://arxiv.org/abs/2203.03466.

[19] Guodong Zhang, Lala Li, Zachary Nado, James Martens, Sushant Sachdeva, George E. Dahl, Christopher J. Shallue, and Roger Grosse. Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL: https://arxiv.org/abs/1907.04164.